

What is a computer system?

- Software

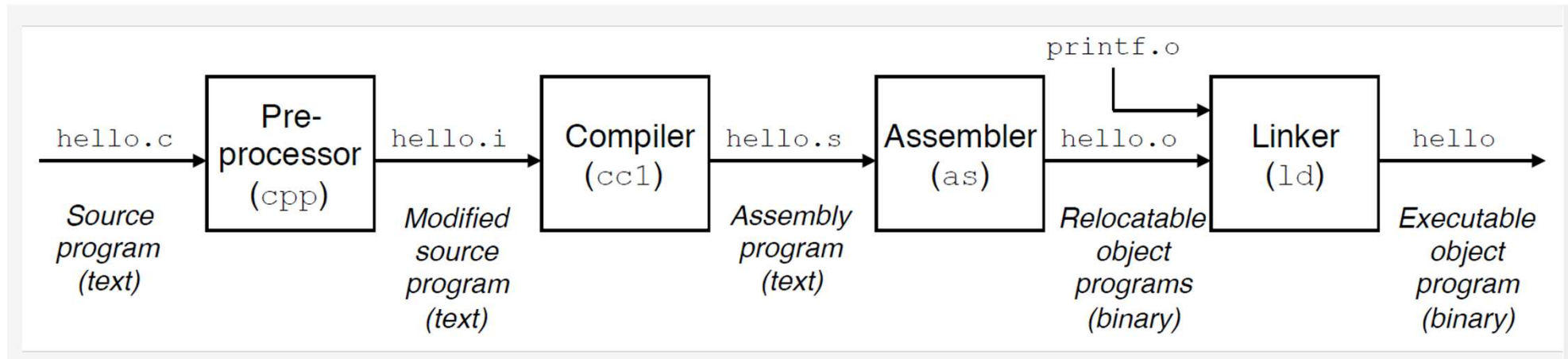
- User programs (instructions and data) to accomplish some tasks
- System software like operating systems

- Hardware

- CPU (registers, ALU, caches, ...)
 - Main memory (DRAM)
 - I/O devices, secondary storage...
- Software written in high-level languages is compiled into binary files (executables) containing instructions that the CPU hardware can execute
 - Operating systems written in high-level language like C
 - Please be comfortable with C before proceeding further in this course

Running a program

- What happens when you run a C program?
 - C code translated into **executable** by compiler in multiple steps (see below)
 - Executable loaded from disk to main memory when program starts
 - CPU fetches program instructions from main memory and executes them



Hardware Organization

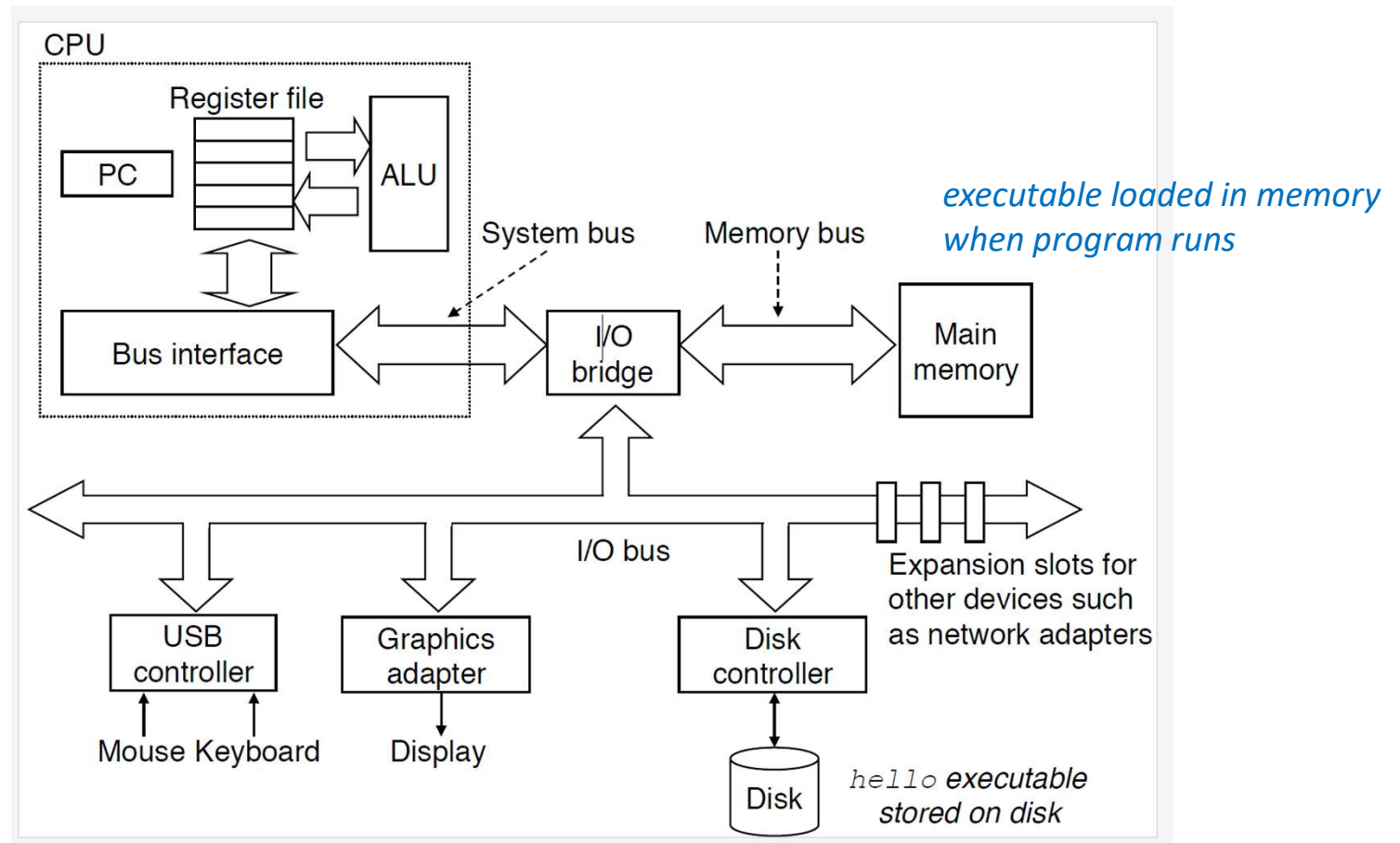


Image credit: CSAPP

CPU ISA

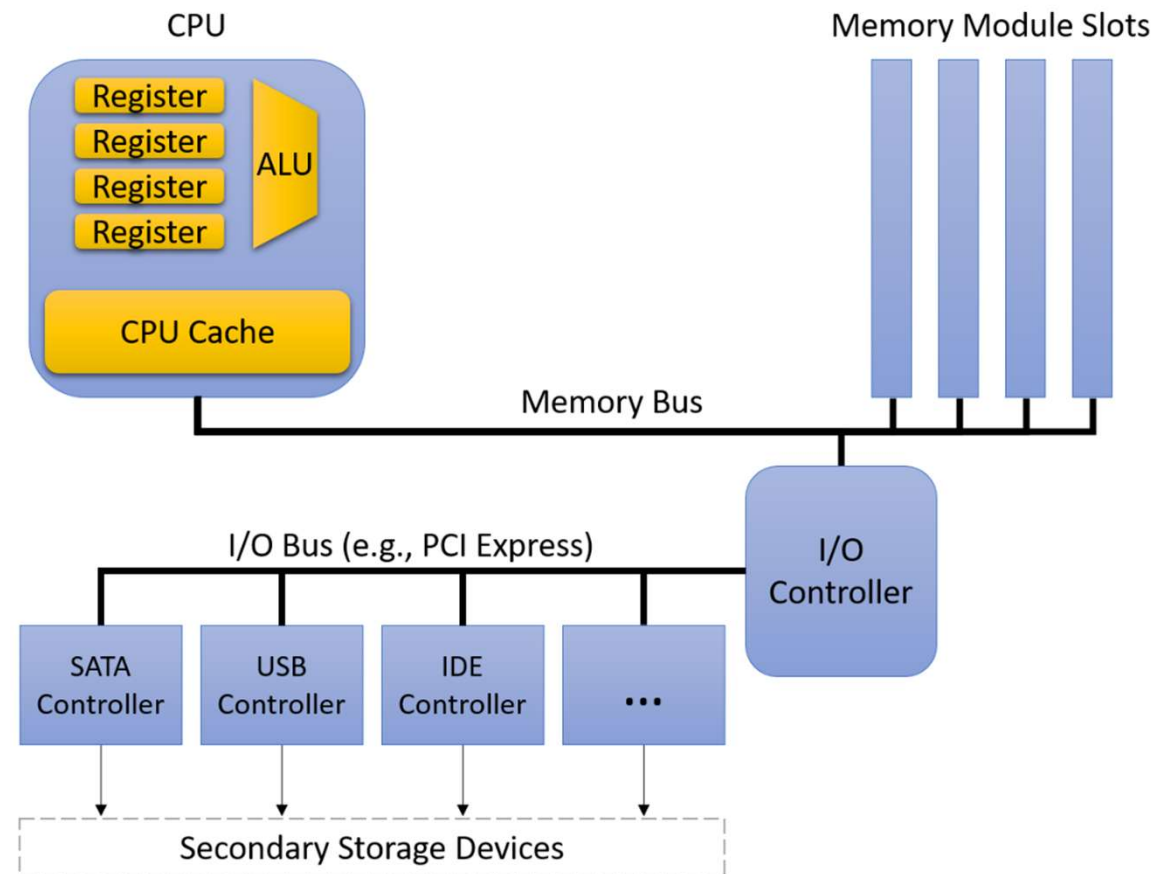
- Every CPU has a well-defined set of
 - **Instructions** that the hardware can execute
 - **Registers** for temporary storage of data within the CPU
- Instructions and registers specified by **ISA** = Instruction Set Architecture
 - Specific to CPU manufacturer (e.g., Intel CPUs follow x86 ISA)
- Registers: special registers (specific purpose) or general purpose
 - **Program counter (PC)** is special register, has memory address of the next instruction to execute on the CPU
 - General purpose registers can be used for anything, e.g., operands in instructions
- Size of registers defined by architecture (32 bit / 64 bit)

CPU instructions

- Some common examples of CPU instructions
 - **Load**: copy content from memory location → register
 - **Store**: copy content from register → memory location
 - **Arithmetic** and **logical** operations like add: $\text{reg1} + \text{reg2} \rightarrow \text{reg3}$, compare, ..
 - **Jump**: change value of PC
 - **Call**: invoke a function
- Simple model of CPU
 - Each clock cycle, **fetch** instruction at PC, **decode**, access required data, **execute**, update PC, repeat
 - PC increments to next instruction, or jumps to some other value
- Many optimizations to this simple model
 - **Pipelining**: run multiple instructions concurrently in a pipeline
 - Many more in modern CPUs to optimize #instructions executed per clock cycle

Memory/storage hierarchy

- Program executable loaded from **secondary storage** to **main memory**
- When CPU runs program, recently accessed instructions and data stored in **CPU caches** (faster access than DRAM)
- **Registers** in CPU provide temporary storage, e.g., hold operands



Memory/storage hierarchy

- Hierarchy of storage elements which store instructions and data
 - CPU registers (small number, accessed in <1 nanosec)
 - Multiple levels of CPU caches (few MB, 1-10 nanosec)
 - Main memory or RAM (few GB, ~100 nanosec)
 - Hard disk (few TB, ~1 millisec)
- Hard disk is non-volatile storage, rest are volatile
 - Hard disk stores files and other data persistently
- As you go down the hierarchy, memory access technology becomes cheaper, slower, less expensive
- CPU caches **transparent** to software, managed by hardware
 - Software only accesses memory, doesn't know if served from cache or DRAM

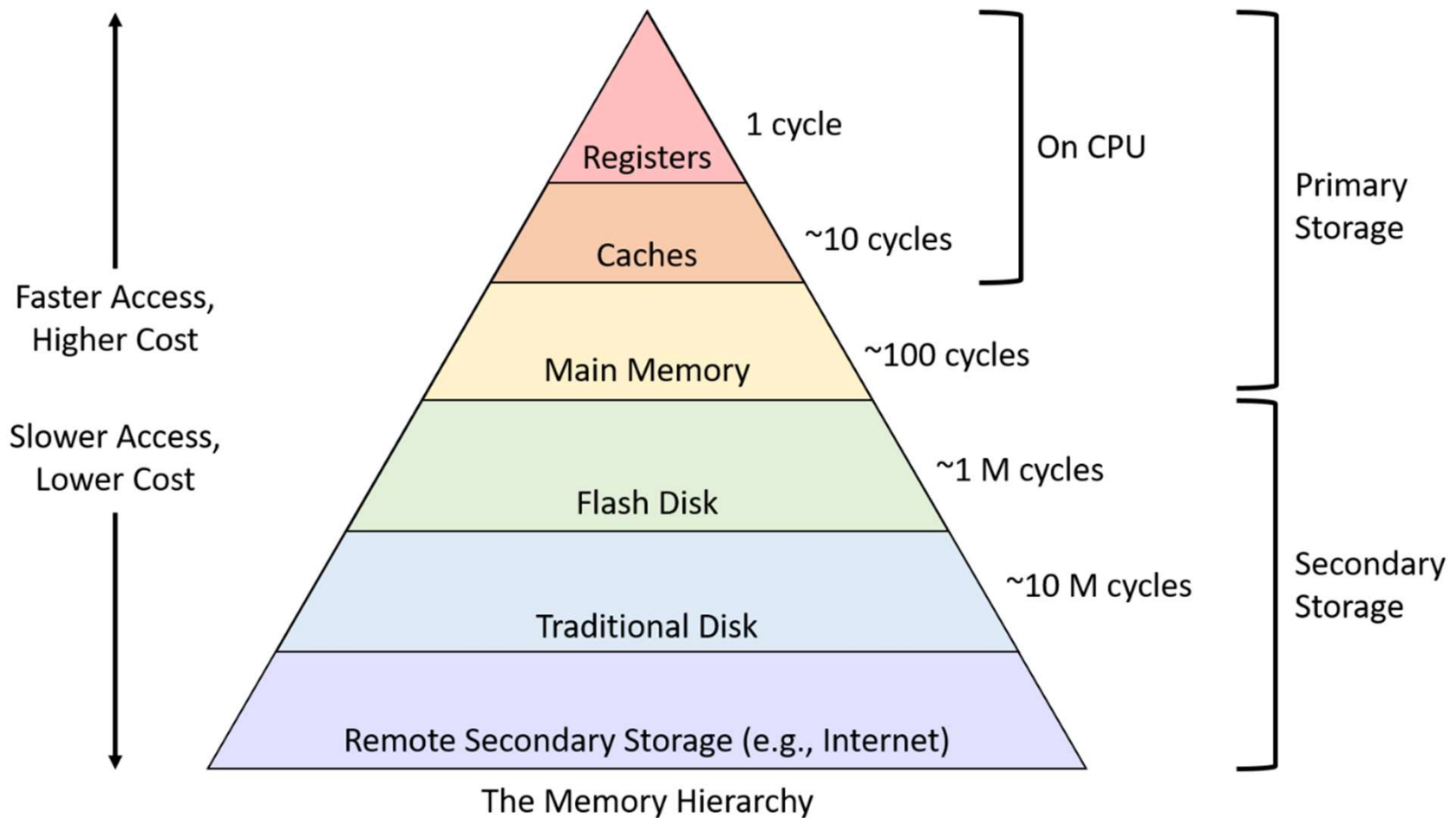


Image credit: Dive Into Systems

Storage Capacity